

Informe del Trabajo Final Integrador: Asistente de Ciberseguridad OT

Fecha de Entrega: 31/08/2026

Autor: Guido Ygounet

Proyecto: [ddsia2026-trabajo-final](#)

La implementación de este asistente busca resolver el acceso ágil a la normativa IEC 62443 en entornos industriales, priorizando la confidencialidad de la red OT en la planta de manufactura mediante inferencia de inteligencia artificial operando íntegramente en el *edge*.

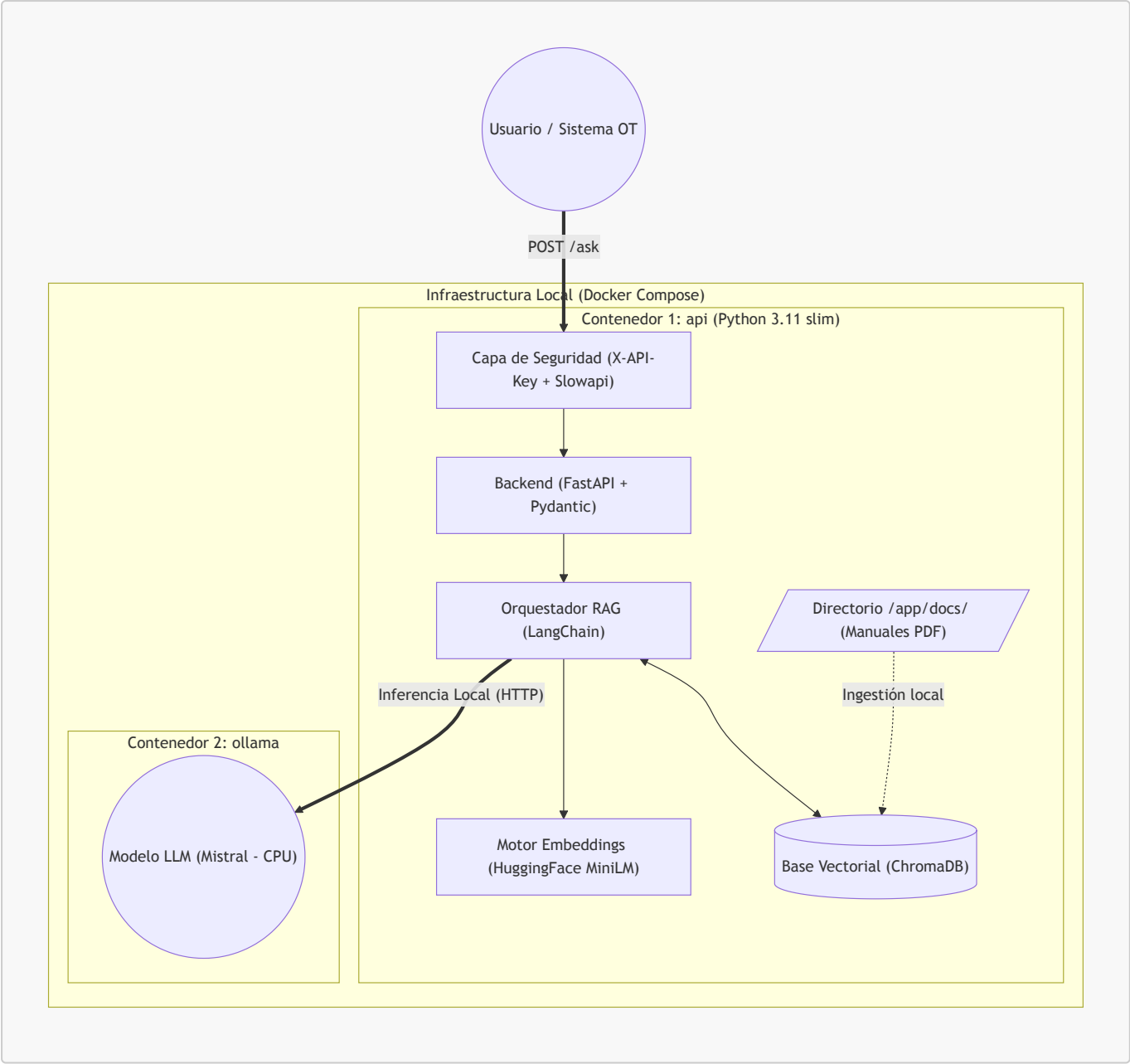
1. Alcance y Cumplimiento de Requisitos

El desarrollo cumple con todos los requerimientos obligatorios de la diplomatura:

- **API HTTP funcional:** Construida con FastAPI, exponiendo un endpoint principal para consultas.
- **RAG Básico:** Implementa ingestión de PDFs locales, embeddings de HuggingFace, recuperación vía ChromaDB y generación con LangChain.
- **LLM Local:** Utiliza el motor Ollama ejecutando el modelo Mistral para evitar la fuga de datos hacia la nube.
- **Contenedorización:** El sistema completo se despliega mediante `docker-compose.yml` e imágenes construidas con `Dockerfile`.
- **Pipeline CI/CD:** Acciones automatizadas en GitHub validan calidad y seguridad estática.
- **Control de Acceso:** El endpoint exige una API Key en las cabeceras HTTP mediante `APIKeyHeader`.
- **Rate Limiting:** Se utiliza la librería Slowapi para restringir el tráfico a 5 peticiones por minuto por IP.
- **Observabilidad:** Los eventos del sistema, como IPs de origen o intentos de inyección, se registran mediante logs estructurados.
- **Modelo de Amenazas:** Análisis exhaustivo utilizando la metodología STRIDE.
- **Guardrails IA:** Validación de esquema con Pydantic y filtrado de palabras clave prohibidas para mitigar inyecciones de *prompt*.
- **Hardening:** Contenedores basados en imágenes `slim` y ejecutados con el usuario no privilegiado `appuser`.

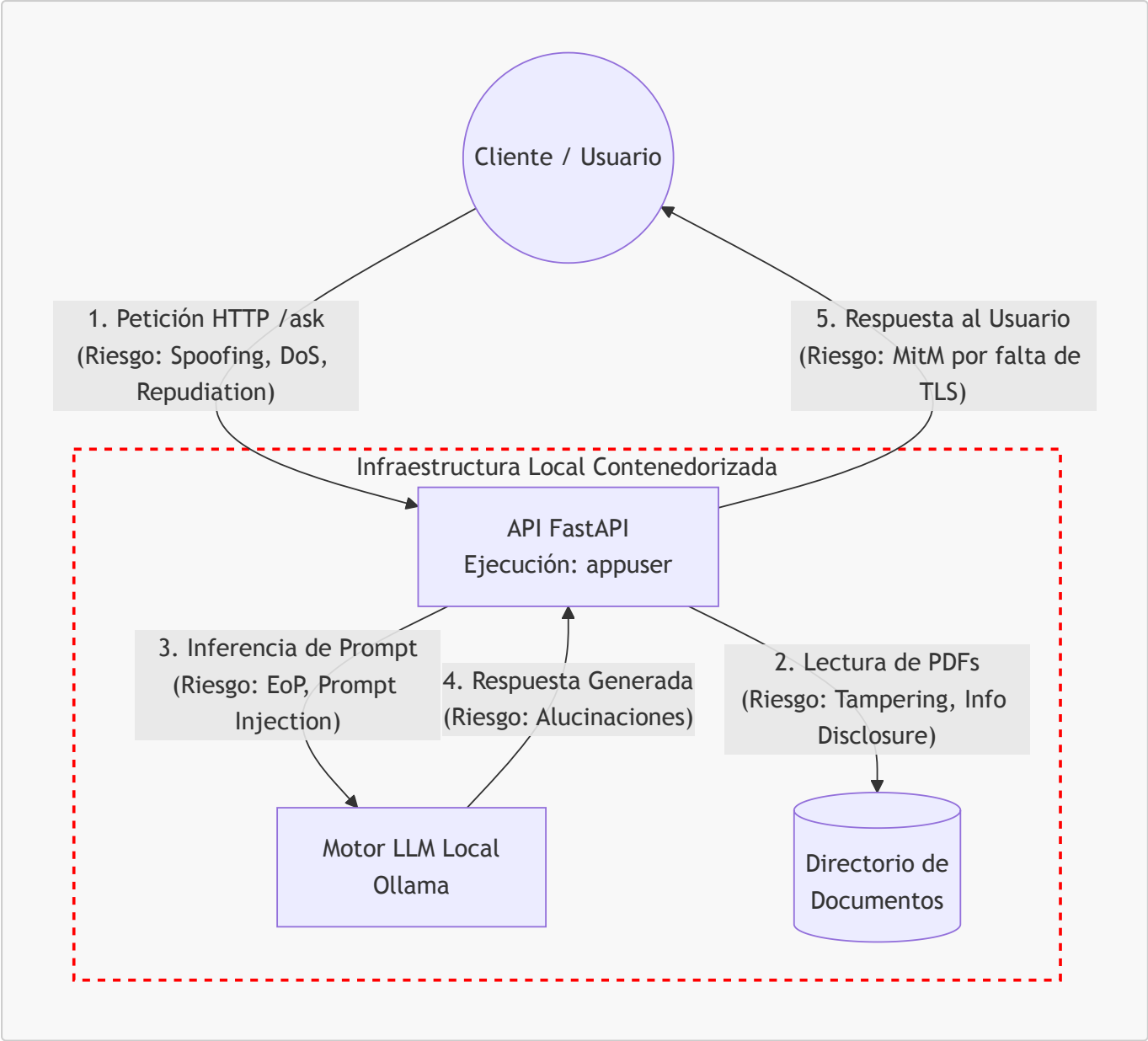
2. Arquitectura del Sistema

La solución opera en un entorno cerrado orquestando dos contenedores principales.



3. Modelo de Amenazas (STRIDE)

El análisis de seguridad contempla las interacciones de los componentes y los vectores de ataque potenciales dentro del límite de confianza del clúster local.



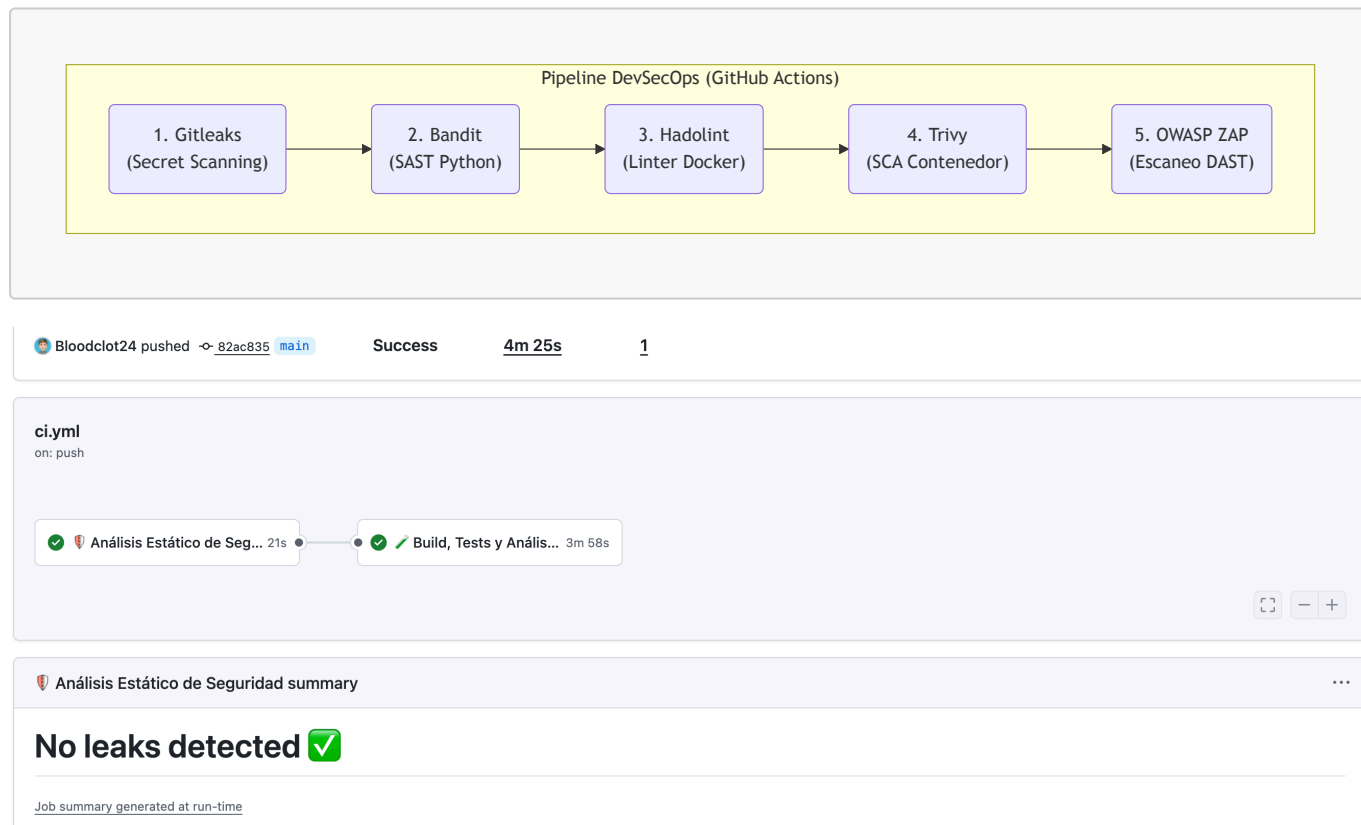
Matriz de Riesgos y Mitigaciones

Categoría	Amenaza Identificada	Mitigación Implementada	Riesgo Residual
Spoofing	Suplantación de identidad para consumir la API.	Exigencia de API Key en los headers validada por FastAPI.	N/A
Tampering	Modificación maliciosa del código o documentos fuente.	Contenedores inmutables y permisos de solo lectura sobre el directorio de documentos.	N/A
Repudiation	Negación de acciones maliciosas.	Logs estructurados registrando IP del cliente y tipo de evento.	N/A

Categoría	Amenaza Identificada	Mitigación Implementada	Riesgo Residual
Information Disclosure	Fuga de datos hacia proveedores externos.	Uso de LLM (Ollama) y embeddings locales asegurando cero exfiltración de datos.	Intercepción por falta de cifrado en tránsito (HTTP sin TLS).
Denial of Service	Agotamiento de recursos locales (CPU/RAM) mediante inundación de peticiones.	Rate Limiting estricto (5 peticiones por minuto) con slowapi.	Agotamiento de recursos por falta de límites a nivel Docker.
Elevation of Privilege	Ejecución de código arbitrario para tomar control del host.	Hardening del contenedor ejecutando procesos como usuario no privilegiado (appuser).	N/A
Riesgos IA	Intentos de Prompt Injection.	Guardrails mediante validación de Pydantic y lista negra de palabras.	Jailbreaks complejos, alucinaciones y envenenamiento de contexto.

4. Flujo DevSecOps

Para garantizar la calidad de la imagen y la prevención de fugas de credenciales en la integración continua, se estructuró un pipeline automatizado.



5. Instrucciones de Despliegue y Pruebas

El despliegue local asume la provisión de las utilidades base de Docker en el servidor de destino.

- **Paso 1:** Almacenar los manuales normativos en formato PDF dentro del directorio `app/docs/`.
- **Paso 2:** Levantar los contenedores de orquestación utilizando el comando `docker-compose up --build -d`.
- **Paso 3:** Descargar los pesos locales del LLM Mistral forzando su tracción manual dentro del contenedor asignado mediante `docker exec -it <nombre_del_contenedor_ollama> ollama run mistral`.
- **Paso 4:** Confirmar la disponibilidad del puerto y enviar peticiones HTTP `POST /ask` incluyendo la cabecera `X-API-Key` y un cuerpo JSON con la llave `question`.
- **Paso 5:** Validar el sistema de seguridad y *guardrails* mediante la ejecución local del conjunto de pruebas a través de `pytest test_main.py -v`.

5.1 API HTTP funcional - Demo

```

$ curl -X POST http://localhost:8080/ask \
  -H "Content-Type: application/json" \
  -H "X-API-Key: " \
  -d '{"question": "¿Cuáles es el paso 11 del owasp top10?"}'
{"answer": "Based on the provided context, the OWASP Top 10 does not have a step 11. The list only goes up to step 10. Here are the 10 steps:\n1. Injection\n2. Broken Aut\nhentication\n3. Sensitive Data Exposure\n4. Insufficient Logging & Monitoring\n5. Broken Access Control\n6. Cross-Site Scripting (XSS)\n7. Insecure Deserialization\n8. Usin\ng Components with Known Vulnerabilities\n9. Insufficient Input Validation\n10. Insufficient Logout\n\nHowever, it seems like there might be a mistake in the context provide\nd, as TA0010 appears twice in the list, which suggests that it could be a typo or an additional threat beyond the Top 10."}

```

6. Detalle de estrategias utilizadas

6.1 API HTTP funcional que utilice un LLM

Se desarrolló una API web utilizando el framework **FastAPI** (Python). El endpoint principal (`POST /ask`) recibe una consulta en formato JSON, la procesa a través del sistema y retorna la respuesta generada por el LLM en formato HTTP estructurado.

6.2. RAG Básico (Ingestión, Embeddings, Recuperación y Generación)

Se implementó un pipeline completo utilizando **LangChain**:

- **Ingestión:** `PyPDFDirectoryLoader` lee la norma ISA/IEC 62443 desde la carpeta local `/docs`.
- **Embeddings:** Se utilizan modelos locales de HuggingFace (`all-MiniLM-L6-v2`).
- **Recuperación:** La vectorización se almacena y consulta en una base de datos **ChromaDB**.
- **Generación:** Se utiliza la cadena `RetrievalQA` de LangChain.

6.3. Uso de LLM (API externa o modelo local)

Para garantizar la confidencialidad de los datos técnicos y cumplir con las mejores prácticas de seguridad OT (Zero Data Exfiltration), se optó por un **modelo local**. Se utiliza el motor **Ollama** ejecutando el modelo *Mistral*, eliminando la dependencia de servicios en la nube y costos por tokens.

6.4. Empaquetado en Contenedor

El sistema entero (API + Base Vectorial) está contenido mediante un `Dockerfile` optimizado. La orquestación junto al motor de Ollama se realiza mediante `docker-compose.yml`, facilitando un despliegue unificado y reproducible.

6.5. Pipeline de CI/CD mínimo (Build, Test y Calidad)

Se implementó un pipeline en **GitHub Actions** ([.github/workflows/ci.yml](#)) con un enfoque DevSecOps que incluye:

- Instalación de dependencias y ejecución de **Tests Unitarios** automatizados (con **pytest** y mocks).
- Escaneo de secretos con **Gitleaks**.
- Análisis estático de código Python (SAST) con **Bandit**.
- Validación de mejores prácticas de empaquetado con **Hadolint**.
- Validación de Build del contenedor y escaneo de vulnerabilidades (SCA) con **Trivy**, bloqueando PRs con vulnerabilidades críticas.

6.6. Autenticación y Control de Acceso en la API

El acceso al endpoint está restringido. Se implementó una verificación mediante **API Key** utilizando el esquema de seguridad nativo de FastAPI (**APIKeyHeader**). Las peticiones sin credenciales válidas reciben un error HTTP 401/403.

6.7. Rate Limiting

Para mitigar el agotamiento de recursos del hardware local (Denial of Wallet / DoS), se integró la librería **Slowapi**. El endpoint **/ask** está estrictamente limitado a **5 peticiones por minuto por IP** cliente.

6.8. Observabilidad Básica (Logs estructurados)

Se utiliza el módulo **logging** de Python configurado para emitir **logs estructurados**. El sistema registra eventos clave con marcas de tiempo, como las direcciones IP de origen, el procesamiento de consultas legítimas (nivel INFO) y la detección de intentos de inyección de código o fallas de autenticación (nivel WARNING/ERROR).

6.9. Modelo de Amenazas Documentado

Se elaboró y adjuntó en el repositorio el archivo **THREAT_MODEL.md**. En él se analizan los componentes del sistema bajo la metodología **STRIDE** (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) detallando las mitigaciones aplicadas en código para cada vector.

6.10. Mitigaciones de Seguridad para IA (Guardrails)

- **Validación de Entradas/Salidas:** A través de esquemas de **Pydantic**, se aplican límites estrictos de longitud y formato en la consulta (mitigando ataques de buffer o estrés en el LLM).
- **Guardrails:** Se implementó un filtro programático básico en el endpoint (lista de palabras prohibidas) que intercepta y bloquea comandos clásicos de *Prompt Injection* ("ignora instrucciones", "system prompt", etc.) antes de que lleguen al modelo.

6.11. Hardening del Contenedor (Sin root e imagen mínima)

- **Imagen Mínima:** Se utiliza **python:3.11-slim** como imagen base, reduciendo drásticamente la superficie de ataque.
- **Ejecución sin privilegios:** El **Dockerfile** crea un usuario específico (**appuser**) y cambia el contexto de ejecución (**USER appuser**) para garantizar que la aplicación y el LLM nunca corran con privilegios de administrador (**root**).

- **Parcheo Dinámico:** El contenedor actualiza sus paquetes base (`apt-get upgrade`) y fuerza las versiones seguras de librerías Python (`msgpack`, `setuptools`) en tiempo de compilación.

7. Resultados obtenidos de las herramientas de seguridad

7.1. Gitleaks

```
name: Checkout del código
uses: actions/checkout@v4
with:
  fetch-depth: 0 # Necesario para que Gitleaks revise todo el
historial

- name: 🕵 Escaneo de Secretos (Gitleaks)
  uses: gitleaks/gitleaks-action@v2
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```


Análisis Estático de Seguridad

succeeded 3 hours ago in 21s






Escaneo de Secretos (Gitleaks)
3s

```

1 node 20 is being deprecated. This workflow is running with node 24 by default. If you need to temporarily use node 20, you can set the
  ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true environment variable. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-
  actions-runners/
2 ▶ Run gitleaks/gitleaks-action@v2
5 (node:2311) [DEP0040] DeprecationWarning: The 'punycode' module is deprecated. Please use a userland alternative instead.
6 (Use 'node --trace-deprecation ...' to show where the warning was created)
7 (node:2311) [DEP0169] DeprecationWarning: 'url.parse()' behavior is not standardized and prone to errors that have security implications. Use the WHATWG URL API instead.
  CVEs are not issued for 'url.parse()' vulnerabilities.
8 [Bloodclot24] is an individual user. No license key is required.
9 gitleaks version: 8.24.3
10 Version to install: 8.24.3 (target directory: /tmp/gitleaks-8.24.3)
11 Cache hit for: gitleaks-cache-8.24.3-linux-x64
12 Received 1518603 of 5712907 (26.6%), 1.4 MBs/sec
13 Received 5712907 of 5712907 (100.0%), 4.8 MBs/sec
14 Cache Size: ~5 MB (5712907 B)
15 /usr/bin/tar -xf /home/runner/work/_temp/c0bf0024-2540-4051-92a2-911a65fab14/cache.tzst -P -C /home/runner/work/ddsia2026-trabajo-final/ddsia2026-trabajo-final --use-
  compress-program unztst
16 Cache restored successfully
17 Gitleaks restored from cache
18 event type: push
19 gitleaks cmd: gitleaks detect --redact -v --exit-code=2 --report-format=sarif --report-path=results.sarif --log-level=debug --log-opts=-1
20 /tmp/gitleaks-8.24.3/gitleaks detect --redact -v --exit-code=2 --report-format=sarif --report-path=results.sarif --log-level=debug --log-opts=-1
21
22   o
23   | \
24   |  o
25   o  ✂
26   ✂   gitleaks
27
28 2:39PM DBG using github.com/wasilibs/go-re2 regex engine
29 2:39PM DBG no gitleaks config found in path .gitleaks.toml, using default gitleaks config
30 2:39PM DBG executing: /usr/bin/git -C . log -p -U0 -1
31 2:39PM DBG SCM platform parsed from host host=github.com platform=github
32 2:39PM INF 1 commits scanned.
33 2:39PM DBG Note: this number might be smaller than expected due to commits with no additions
34 2:39PM INF scanned ~55 bytes (55 bytes) in 144ms
35 2:39PM INF no leaks found

```

7.2. Bandit

```
name: 🔍 Escaneo SAST para Python (Bandit)
run: |
  pip install bandit
  # Escanea la carpeta app/ buscando fallos de seguridad en el código
  bandit -r app/ -ll -ii
```


Análisis Estático de Seguridad

succeeded 3 hours ago in 21s







Escaneo SAST para Python (Bandit)

3s

```

24 Downloading pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)
25 Collecting mdurl~=0.1 (from markdown-it-py>=2.2.0->rich->bandit)
26 Downloading mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
27 Downloading bandit-1.9.4-py3-none-any.whl (134 kB)
28 Downloading pyyaml-6.0.3-cp311-cp311-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (806 kB)
29 _____ 806.6/806.6 kB 117.1 MB/s 0:00:00
30 Downloading stevedore-5.9.1-py3-none-any.whl (54 kB)
31 Downloading rich-15.0.0-py3-none-any.whl (310 kB)
32 Downloading pygments-2.21.0-py3-none-any.whl (1.3 MB)
33 _____ 1.3/1.3 MB 135.0 MB/s 0:00:00
34 Downloading markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
35 Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
36 Installing collected packages: stevedore, PyYAML, pygments, mdurl, markdown-it-py, rich, bandit
37
38 Successfully installed PyYAML-6.0.3 bandit-1.9.4 markdown-it-py-4.2.0 mdurl-0.1.2 pygments-2.21.0 rich-15.0.0 stevedore-5.9.1
39 [main] INFO profile include tests: None
40 [main] INFO profile exclude tests: None
41 [main] INFO cli include tests: None
42 [main] INFO cli exclude tests: None
43 [main] INFO running on Python 3.11.16
44 Run started:2026-08-29 14:39:44.479916+00:00
45
46 Test results:
47     No issues identified.
48
49 Code scanned:
50     Total lines of code: 108
51     Total lines skipped (#nosec): 0
52     Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0
53
54 Run metrics:
55     Total issues (by severity):
56         Undefined: 0
57         Low: 11
58         Medium: 0
59         High: 0
60     Total issues (by confidence):
61         Undefined: 0
62         Low: 0
63         Medium: 0
64         High: 11
65 Files skipped (0):
```

7.3. Hadolint

```
name: 🐳 Linting del Dockerfile (Hadolint)
uses: hadolint/hadolint-action@v3.1.0
with:
  dockerfile: Dockerfile
```


 **Análisis Estático de Seguridad**

succeeded 3 hours ago in 21s

Q Search logs

>  Set up job

>  Build hadolint/hadolint-action@v3.1.0

>  Checkout del código

>   Escaneo de Secretos (Gitleaks)

>   Configurar Python

>   Escaneo SAST para Python (Bandit)

>   Linting del Dockerfile (Hadolint)

2s

7s

1s

3s

0s

3s

0s

1 ▶ Run hadolint/hadolint-action@v3.1.0

18 /usr/bin/docker run --name ace9e558b4e840add4c44856b8d61e8fd8e99_d930fe --label 2ace9e --workdir /github/workspace --rm -e "pythonLocation" -e "PKG_CONFIG_PATH" -e "Python_ROOT_DIR" -e "Python2_ROOT_DIR" -e "Python3_ROOT_DIR" -e "LD_LIBRARY_PATH" -e "INPUT_DOCKERFILE" -e "INPUT_CONFIG" -e "INPUT_RECURSIVE" -e "INPUT_OUTPUT-FILE" -e "INPUT_NO-COLOR" -e "INPUT_NO-FAIL" -e "INPUT_VERBOSE" -e "INPUT_FORMAT" -e "INPUT_FAILURE-THRESHOLD" -e "INPUT_OVERRIDE-ERROR" -e "INPUT_OVERRIDE-WARNING" -e "INPUT_OVERRIDE-INFO" -e "INPUT_OVERRIDE-STYLE" -e "INPUT_IGNORE" -e "INPUT_TRUSTED-REGISTRIES" -e "NO_COLOR" -e "HADOLINT_NOFAIL" -e "HADOLINT_VERBOSE" -e "HADOLINT_FORMAT" -e "HADOLINT_FAILURE_THRESHOLD" -e "HADOLINT_OVERRIDE_ERROR" -e "HADOLINT_OVERRIDE_WARNING" -e "HADOLINT_OVERRIDE_INFO" -e "HADOLINT_OVERRIDE_STYLE" -e "HADOLINT_IGNORE" -e "HADOLINT_TRUSTED_REGISTRIES" -e "HADOLINT_CONFIG" -e "HADOLINT_RECURSIVE" -e "HADOLINT_OUTPUT" -e "HOME" -e "GITHUB_JOB" -e "GITHUB_REF" -e "GITHUB_SHA" -e "GITHUB_REPOSITORY" -e "GITHUB_REPOSITORY_OWNER" -e "GITHUB_REPOSITORY_OWNER_ID" -e "GITHUB_RUN_ID" -e "GITHUB_RUN_NUMBER" -e "GITHUB_RETENTION_DAYS" -e "GITHUB_RUN_ATTEMPT" -e "GITHUB_ACTOR_ID" -e "GITHUB_ACTOR" -e "GITHUB_WORKFLOW" -e "GITHUB_HEAD_REF" -e "GITHUB_BASE_REF" -e "GITHUB_EVENT_NAME" -e "GITHUB_SERVER_URL" -e "GITHUB_API_URL" -e "GITHUB_GRAPHQL_URL" -e "GITHUB_REF_NAME" -e "GITHUB_REF_PROTECTED" -e "GITHUB_REF_TYPE" -e "GITHUB_WORKFLOW_REF" -e "GITHUB_WORKFLOW_SHA" -e "GITHUB_REPOSITORY_ID" -e "GITHUB_TRIGGERING_ACTOR" -e "GITHUB_WORKSPACE" -e "GITHUB_ACTION" -e "GITHUB_EVENT_PATH" -e "GITHUB_ACTION_REPOSITORY" -e "GITHUB_ACTION_REF" -e "GITHUB_PATH" -e "GITHUB_ENV" -e "GITHUB_STEP_SUMMARY" -e "GITHUB_STATE" -e "GITHUB_OUTPUT" -e "GITHUB_ARTIFACTS" -e "GITHUB_ARTIFACTS_LIST" -e "RUNNER_OS" -e "RUNNER_ARCH" -e "RUNNER_NAME" -e "RUNNER_ENVIRONMENT" -e "RUNNER_TOOL_CACHE" -e "RUNNER_TEMP" -e "RUNNER_WORKSPACE" -e "ACTIONS_RUNTIME_URL" -e "ACTIONS_RUNTIME_TOKEN" -e "ACTIONS_CACHE_URL" -e "ACTIONS_RESULTS_URL" -e "ACTIONS_ORCHESTRATION_ID" -e GITHUB_ACTIONS=true -e CI=true -v "/var/run/docker.sock":"/var/run/docker.sock" -v "/home/runner/work/_temp":"/github/runner_temp" -v "/home/runner/work/_temp/github_home":"/github/home" -v "/home/runner/work/_temp/github_workflow":"/github/workflow" -v "/home/runner/work/_temp/runner_file_commands":"/github/file_commands" -v "/home/runner/work/ddsia2026-trabajo-final/ddsia2026-trabajo-final":"/github/workspace" 2ace9e:558b4e840add4c44856b8d61e8fd8e99 "Dockerfile"

19

20 Hadolint output saved to: /dev/stdout

7.4. Pytest

name:  Ejecutar Tests Unitarios

env: PYTHONPATH: .

run: python -m pytest -v -W ignore

 **Build, Tests y Análisis de Contenedor**

succeeded 3 hours ago in 3m 58s

Q Search logs

>  Set up job

>  Checkout del código

>  Configurar Python

>  Crear carpeta docs temporal

>  Instalar dependencias para testing

>   Ejecutar Tests Unitarios

2s

1s

0s

0s

1m 59s

2s

1 ▶ Run python -m pytest -v -W ignore

12 ===== test session starts =====

13 platform linux -- Python 3.11.16, pytest-9.1.1, pluggy-1.6.0 -- /opt/hostedtoolcache/Python/3.11.16/x64/bin/python

14 cachedir: .pytest_cache

15 rootdir: /home/runner/work/ddsia2026-trabajo-final/ddsia2026-trabajo-final

16 plugins: anyio-4.14.2, mock-3.15.1, langsmith-0.11.2

17 collecting ... collected 6 items

18

19 app/test_main.py::test_read_root PASSED [16%]

20 app/test_main.py::test_ask_missing_api_key PASSED [33%]

21 app/test_main.py::test_ask_invalid_api_key PASSED [50%]

22 app/test_main.py::test_prompt_injection_guardrail PASSED [66%]

23 app/test_main.py::test_input_validation_min_length PASSED [83%]

24 app/test_main.py::test_successful_ask PASSED [100%]

25

26 ===== 6 passed in 0.16s =====

7.5. Trivy

9 / 11

```
name: 🚫 Escaneo de Vulnerabilidades del Contenedor (Trivy)
uses: aquasecurity/trivy-action@master
with:
  image-ref: 'ddsia-api-local:latest'
  format: 'table'
  exit-code: '1' # Hace que el pipeline falle si encuentra algo
  ignore-unfixed: true
  vuln-type: 'os,library'
  severity: 'CRITICAL' # Solo reportar vulnerabilidades Críticas
```

Build, Tests y Análisis de Contenedor

succeeded 3 hours ago in 3m 58s

Search logs

🔄 ⚙️

📄 🚫 Escaneo de Vulnerabilidades del Contenedor (Trivy)

32s

1 ▶ Run aquasecurity/trivy-action@master

24 ▶ 🟢 Install Trivy

104 ▶ 🟢 Get current date

115 ▶ 🟢 Restore DB from cache

139 ▶ 🟢 Set GitHub Path

151 ▶ 🟢 Clear Trivy Envs file

163 ▶ 🟢 Set Trivy environment variables

239 ▶ 🟢 Run Trivy

240 ▶ Run entrypoint.sh

257 ▶ Running Trivy with options: trivy image ddsia-api-local:latest

258 ▶

1s

0s

2s

0s

0s

0s

0s

28s

Resumen del Escaneo de Seguridad (Trivy)

- **Imagen Evaluada:** ddsia-api-local:latest (Debian 13.6)
- **Alcance del Escaneo:** Sistema operativo base y 85 dependencias de Python.
- **Vulnerabilidades Detectadas:** 0
- **Secretos Expuestos:** 0
- **Estado Final:** 🟢 Aprobado (Clean)

Objetivo Evaluado	Tipo de Artefacto	Vulnerabilidades	Secretos	Resultado
ddsia-api-local:latest	OS (Debian 13.6)	0	Ninguno	✅ Limpio
Entorno Python 3.11	python-pkg (85 librerías)	0	Ninguno	✅ Limpio

7.6 ZAP

```
docker run --rm -v $(pwd):/zap/wrk/:rw --network="host" -t zaproxy/zap-stable zap-full-scan.py -t http://localhost:8080 -r owasp-report.html
```

ZAP by
Checkmarx

ZAP Scanning Report

Sites: <https://host.docker.internal:8080> <http://host.docker.internal:8080>

Generated on Fri, 28 Aug 2026 01:54:34

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	2
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	4
info	Informational		Percentage of network failures	1 %
info	Informational	http://host.docker.internal:8080	Percentage of responses with status code 2xx	23 %
info	Informational	http://host.docker.internal:8080	Percentage of responses with status code 4xx	76 %
info	Informational	http://host.docker.internal:8080	Percentage of endpoints with content type application/json	100 %
info	Informational	http://host.docker.internal:8080	Percentage of endpoints with method GET	100 %
info	Informational	http://host.docker.internal:8080	Count of total endpoints	3
info	Informational	https://host.docker.internal:8080	Percentage of endpoints with method GET	100 %
info	Informational	https://host.docker.internal:8080	Count of total endpoints	1

Alerts

Name	Risk Level	Number of Instances
HTTP Only Site	Medium	1
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	2
X-Content-Type-Options Header Missing	Low	2
Storable and Cacheable Content	Informational	4